registers on the first loop (lines 004 and 005), and another small section of code to finish the last loop (lines 010 and 011).

To understand how this works, study line 008, the only statement inside the loop. In this single statement, *four* operations are being carried out in parallel: (1) the value for $x[\ ]$ is moved from a circular buffer in program memory and placed in f2; (2) the value for $y[\ ]$ is being moved from a circular buffer in data memory and placed in f4; (3) the previous values of f2 and f4 are multiplied and placed in f8; and (4) the previous value in f8 is added to the accumulator in f12.

For example, the fifth time that line 008 is executed, $x[7]$ and $y[7]$ are fetched from memory and stored in f2 and f4. At the same time, the values for $x[6]$ and $y[6]$ (that were in f2 and f4 at the start of this cycle) are multiplied and placed in f8. In addition, the value of $x[5] \times y[5]$ (that was in f8 at the start of this cycle) is added to the value of f12.

Let's compare the number of clock cycles required by the unoptimized and the optimized programs. Keep in mind that there are 20 loops, with four actions being required in each loop. The unoptimized program requires 80 clock cycles to carry out the actions within the loops, plus 5 clock cycles of overhead, for a total of 85 clock cycles. In comparison, the optimized program conducts 18 loops in 18 clock cycles, but requires 11 clock cycles of overhead to prime the registers and complete the last loop. This results in a total execution time of 29 clock cycles, or about three times faster than the brute force method.

Here is the big question: How fast does the C program execute relative to the assembly code? When the program in Table 28-2 is compiled, does the executable code resemble our *efficient* or *inefficient* assembly example? The answer is that the compiler generates the *efficient* code. However, it is important to realize that the dot product is a very simple example. The compiler has a much more difficult time producing optimized code when the program becomes more complicated, such as multiple nested loops and erratic jumps to subroutines. If you are doing something straightforward, expect the compiler to provide you a nearly optimal solution. If you are doing something strange or complicated, expect that an assembly program will execute significantly faster than one written in C. In the worst case, think a factor of 2-3. As previously mentioned, the efficiency of C versus assembly depends greatly on the particular DSP being used. Floating point architectures can generally be programmed more efficiently than fixed-point devices when using high-level languages such as C. Of course, the proper software tools are important for this, such as a debugger with profiling features that help you understand how long different code segments take to execute.

There is also a way you can get the best of both worlds: write the program in C, but use assembly for the critical sections that must execute quickly. This is one reason that C is so popular in science and engineering. It operates as a high-level language, but also allows you to directly manipulate